

AWS REST API Lab Part 3: Test the Full Workflow with Postman

To Begin with the Lab

Summary of the Lab

In this final part of the **REST API** lab, the complete workflow was tested using **Postman** as the frontend client. The lab validated the three REST API operations that were built earlier: creating a user with POST, reading a user with GET, and deleting a user with DELETE. Each request was sent to the API Gateway invoke URL for the **dev** stage, and the results were verified in **Amazon DynamoDB**. The POST request inserted a new item into **UsersTable**, the GET request returned the stored user record, and the DELETE request removed it successfully. The lab ended by reminding that the **REST API**, Lambda functions, and DynamoDB table should be kept for upcoming labs.

Understanding the Final REST API Test

This step exists so you can confirm the whole application works from outside AWS, just like a real client app would. **Postman** sends HTTP requests to **API Gateway**, which then forwards them to the correct **AWS Lambda** function. That confirms the REST resources, methods, Lambda integration, and DynamoDB operations are all connected properly. It also shows why **API Gateway** is needed: the frontend should not call Lambda directly, and the gateway provides the secure public entry point.

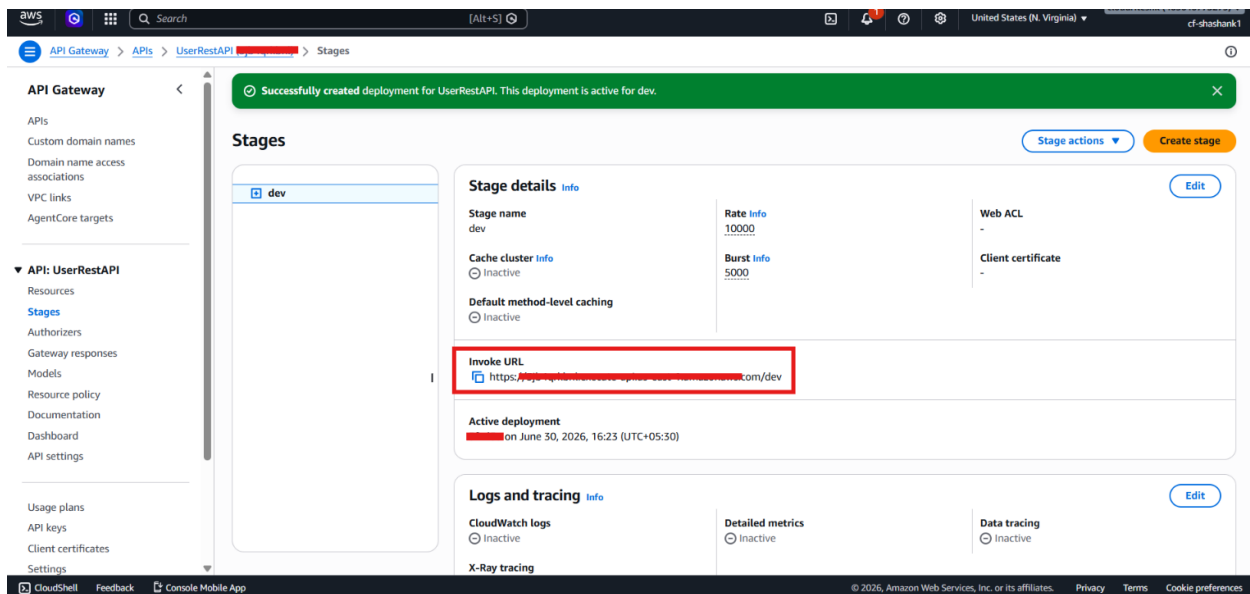
Example:

A client wants to create a user named Kabir, read user 101, and then delete the same record. It sends POST /user to create the record, GET /user/101 to fetch it, and DELETE /user/101 to remove it. Each request is routed by **API Gateway** to the appropriate Lambda function, which performs the DynamoDB operation.

Lab Steps

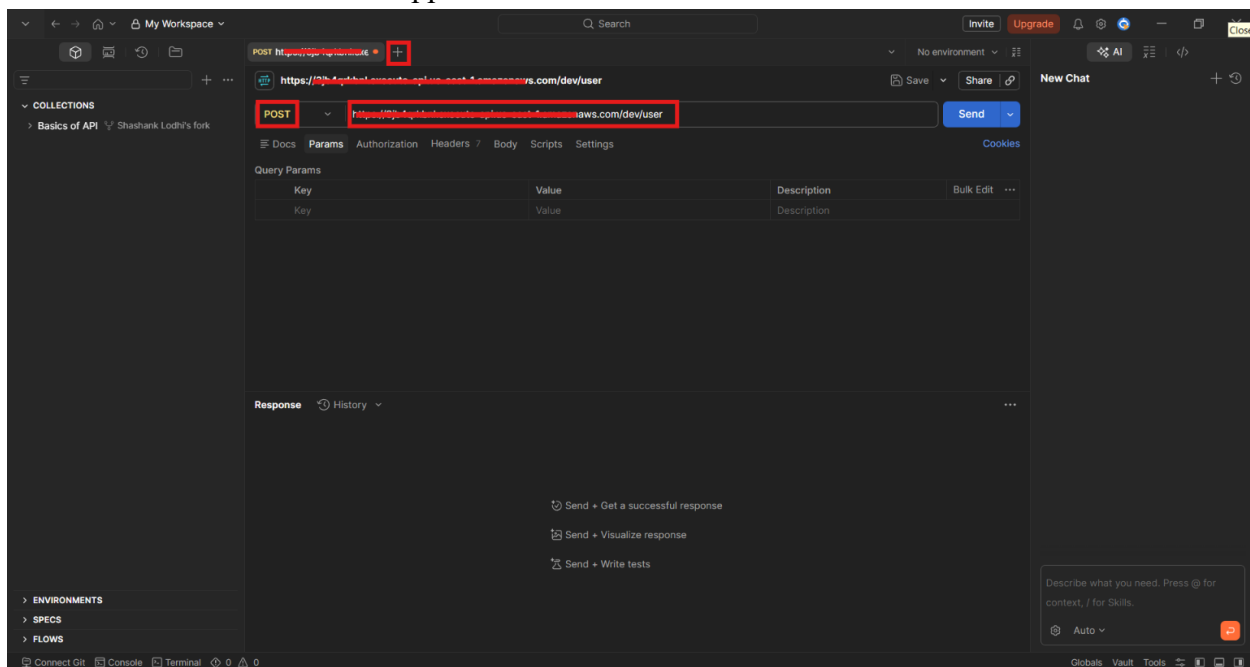
Step 1 – Open the Invoke URL

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Select the REST API.
- Open **Stages**.
- Copy the **Invoke URL** for the dev stage.



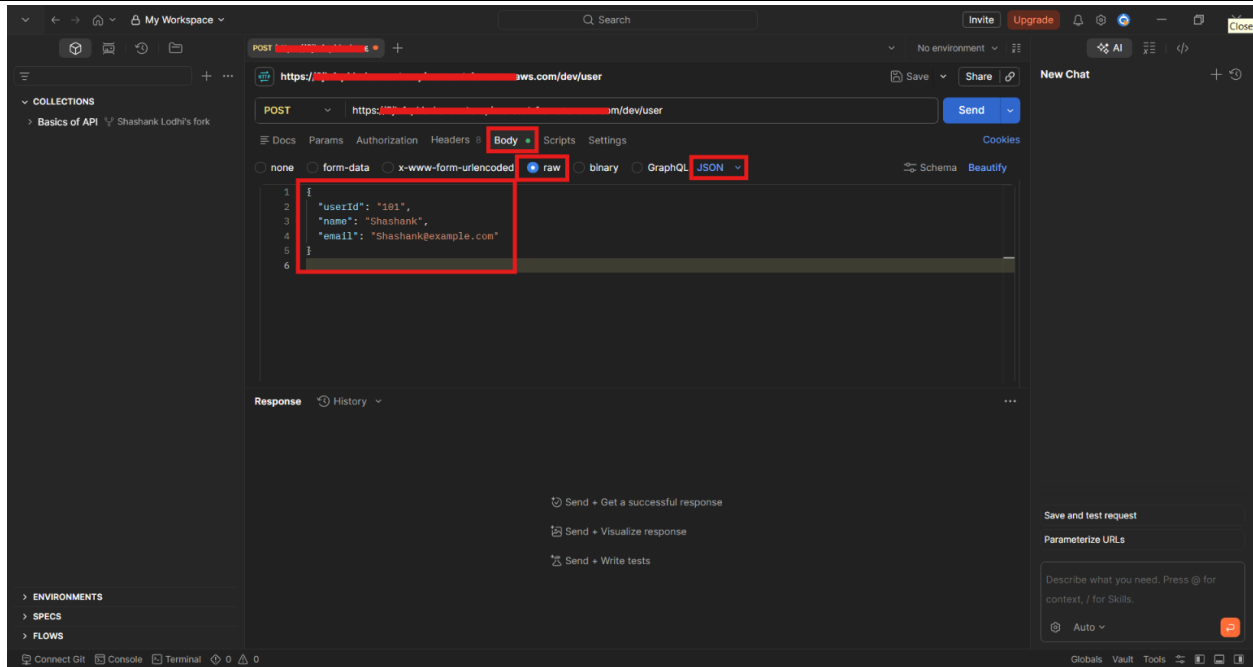
Step 2 – Test the Create User Request

- Open **Postman**.
- Create a new HTTP request.
- Set the method to **POST**.
- Paste the invoke URL and append **/user**.

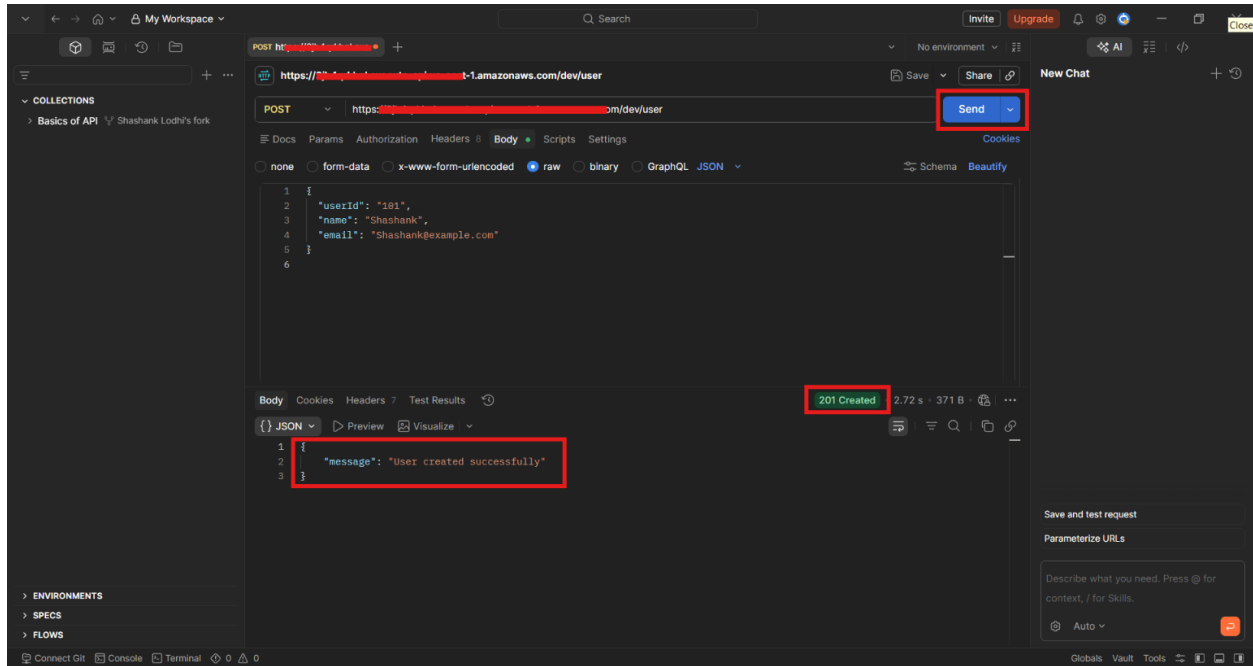


- Open the **Body** tab.
- Select **raw**.
- Choose **JSON**.
- Add the request body.

```
{
  "userId": "101",
  "name": "Shashank",
  "email": "Shashank@example.com"
}
```



- Click Send.
- Confirm that the response shows user creation success.



- Open Amazon DynamoDB.

- Go to **Tables** → **UsersTable**.
- Click **Explore table items**.

The screenshot shows the AWS DynamoDB console interface. On the left, the 'DynamoDB' sidebar is visible with options like Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The 'Explore items' option is selected, leading to the 'UsersTable' view. In the center, the 'Tables (4)' list shows 'UsersTable' selected. The right panel displays the 'UsersTable' details, including a 'Scan or query items' section with a 'Scan' button. Below this, a green status bar indicates 'Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2'. The 'Table: UsersTable - Items returned (1)' section shows a single record with the following details:

userid (String)	email	name
101	Shashank@...	Shashank

- Confirm that the new record was inserted.

Step 3 – Test the Get User Request

- In **Postman**, change the method to **GET**.
- Use the path `/user/101`.
- Click **Send**.
- Confirm that the user record is returned.

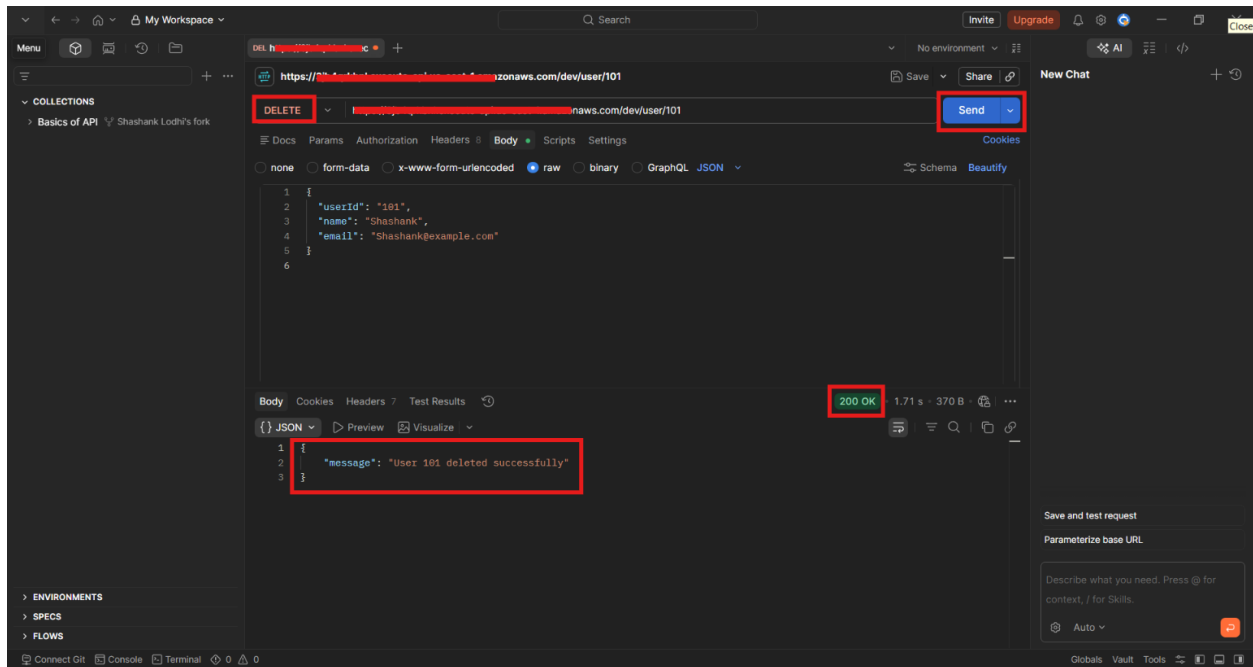
The screenshot shows the Postman API client interface. The 'GET' method is selected for the request to 'https://api.example.com/dev/user/101'. The 'Send' button is highlighted. The response is shown in the 'Body' tab, indicating a '200 OK' status. The response body is a JSON object with the following details:

```
{
  "email": "Shashank@example.com",
  "name": "Shashank",
  "userId": "101"
}
```

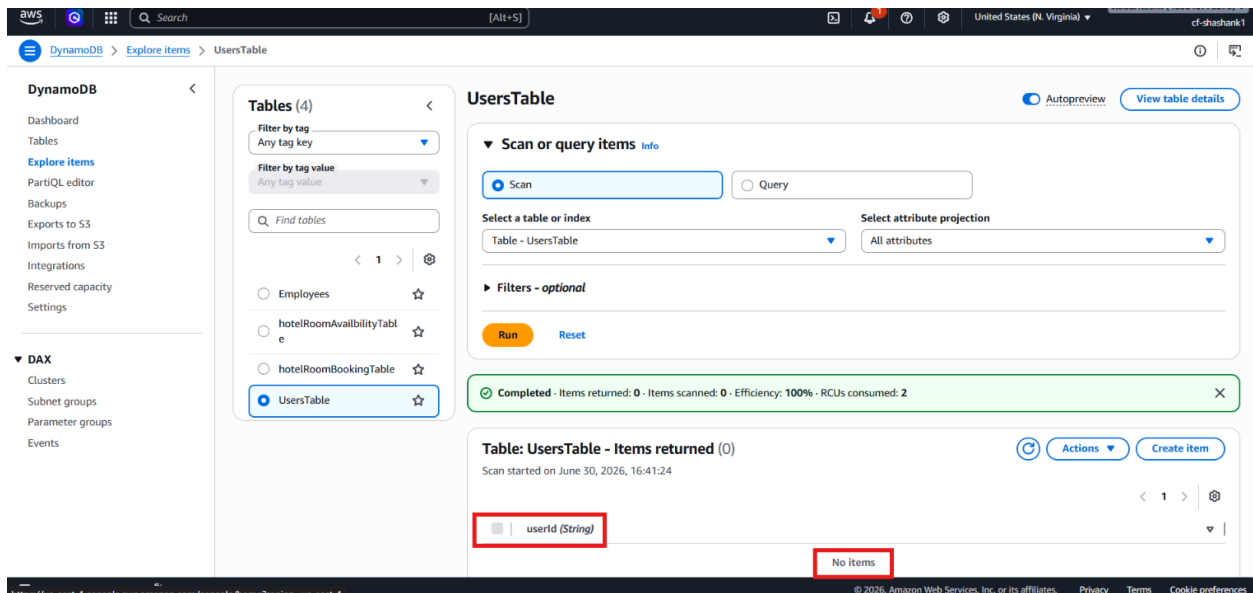
- Verify that the response matches the record stored in **UsersTable**.

Step 4 – Test the Delete User Request

- In **Postman**, change the method to **DELETE**.
- Use the path `/user/101`.
- Click **Send**.
- Confirm that the response shows deletion success.



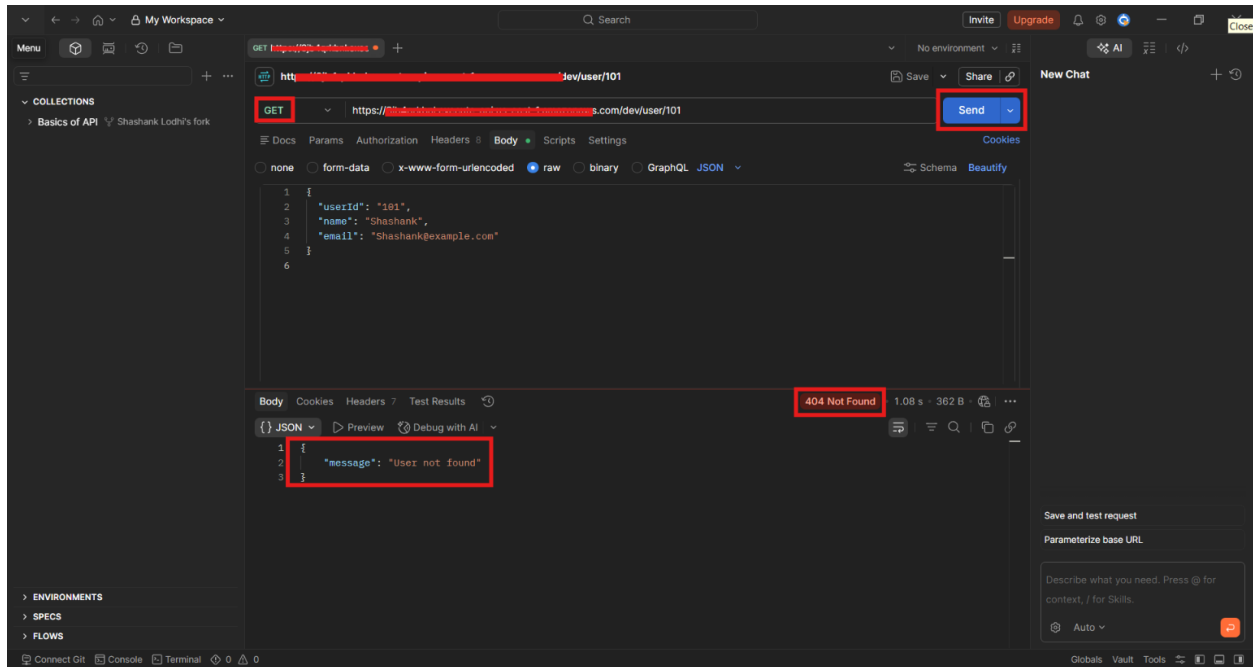
- Return to **Amazon DynamoDB**.
- Refresh **UsersTable**.



- Confirm that the record was removed.

Step 5 – Verify the Missing Record Case

- Send the same GET /user/101 request again after deletion.
- Confirm that the response returns **User not found**.



- Verify that the API handles missing records correctly.

Verification

- The REST API workflow was tested successfully using **Postman**.
- The POST route created a record in **UsersTable**.
- The GET route returned the stored user record.
- The DELETE route removed the record successfully.
- The missing-record case returned **User not found**.
- The full request flow from client → **API Gateway** → **AWS Lambda** → **Amazon DynamoDB** was verified.